



Inter Process Communication (IPC)

Ha Noi August - 2023

OPEN MIND - OPEN DOORS



Contents

1. Introduction to Inter Process Communication (IPC)

2. IPC Methods in Android

3. AIDL (Android Interface Definition Language)

4. Messenger

5. Demo

1. Introduction to IPC

In the Android operating system environment, applications typically run in separate processes. However, there are cases when applications need to share data, information, or interact with each other. This is where Inter Process Communication (IPC) becomes important. IPC allows applications to interact and exchange data safely and efficiently.

2. IPC Methods in Android

2.1. Intent

- ❖ Intent is a simple mechanism for sending messages between different components of an application, and even between different applications.
- ❖ It's primarily used to trigger activities or services and send basic data through Bundles.

2.2. Broadcasts

- ❖ Broadcasts enable an application to send messages to all other applications on the system.
- ❖ Applications can register to listen for broadcasts and perform appropriate actions when they receive them.

2.3. Content Providers

- ❖ Content Providers allow an application to share data with other applications through a querying mechanism.
- ❖ They are mainly used for sharing structured data, such as data from an SQLite database.

2. IPC Methods in Android

2.4. Bound Services(Binder)

- ❖ Bound Services are services that are bound to and called by the application.
- ❖ They allow applications to interact with services and pass data through a bound interface.

2.5. AIDL (Android Interface Definition Language)

- ❖ AIDL is a mechanism that enables communication between processes by defining service interfaces.
- ❖ It's typically used when complex data needs to be passed between processes.

2.6. Messenger

- ❖ The Messenger is another IPC method in Android that relies on using Handler to send and receive messages between different processes.
- ❖ It's mainly used for sending simple messages between components of an application or between different applications.

2. IPC - AIDL (Android Interface Definition Language)

2.5. AIDL (Android Interface Definition Language)

Before you begin designing your AIDL interface, be aware that calls to an AIDL interface are direct function calls. Don't make assumptions about the thread in which the call occurs. What happens is different depending on whether the call is from a thread in the local process or a remote process:

- Calls made from the **local process** execute in the same thread that is making the call. If this is your main UI thread, that thread continues to execute in the AIDL interface. If it is another thread, that is the one that executes your code in the service. Thus, if only local threads are accessing the service, you can completely control which threads are executing in it. But if that is the case, don't use AIDL at all; instead, create the interface by implementing a Binder.
- Calls from a **remote process** are dispatched from a thread pool the **platform maintains inside your own process**. Be prepared for incoming calls from unknown threads, with multiple calls happening at the same time. In other words, an implementation of an AIDL interface must be completely thread-safe. Calls made from one thread on the same remote object arrive in order on the receiver end.
- The **oneway** keyword modifies the behavior of remote calls. When it is used, a remote call does not block. It sends the transaction data and immediately returns. The implementation of the interface eventually receives this as a regular call from the Binder thread pool as a normal remote call. If oneway is used with a local call, there is no impact, and the call is still synchronous.

2. IPC - AIDL (Android Interface Definition Language)

2.5. AIDL (Android Interface Definition Language)

Defining an AIDL interface

Define your AIDL interface in an **.aidl** file using the Java programming language syntax, then save it in the source code, in the **src/ directory**, of both the application hosting the service and any other application that binds to the service.

When you build each application that contains the **.aidl** file, the Android SDK tools generate an **IBinder** interface based on the **.aidl** file and save it in the project's **gen/ directory**. The service must implement the **IBinder** interface as appropriate. The client applications can then bind to the service and call methods from the **IBinder** to perform IPC.

To create a bounded service using AIDL, follow these steps, which are described in the sections that follow:

- **Create the .aidl file**

This file defines the programming interface with method signatures.

- **Implement the interface**

The Android SDK tools generate an interface in the Java programming language based on your **.aidl** file. This interface has an inner abstract class named **Stub** that extends **Binder** and implements methods from your AIDL interface. You must extend the **Stub** class and implement the methods.

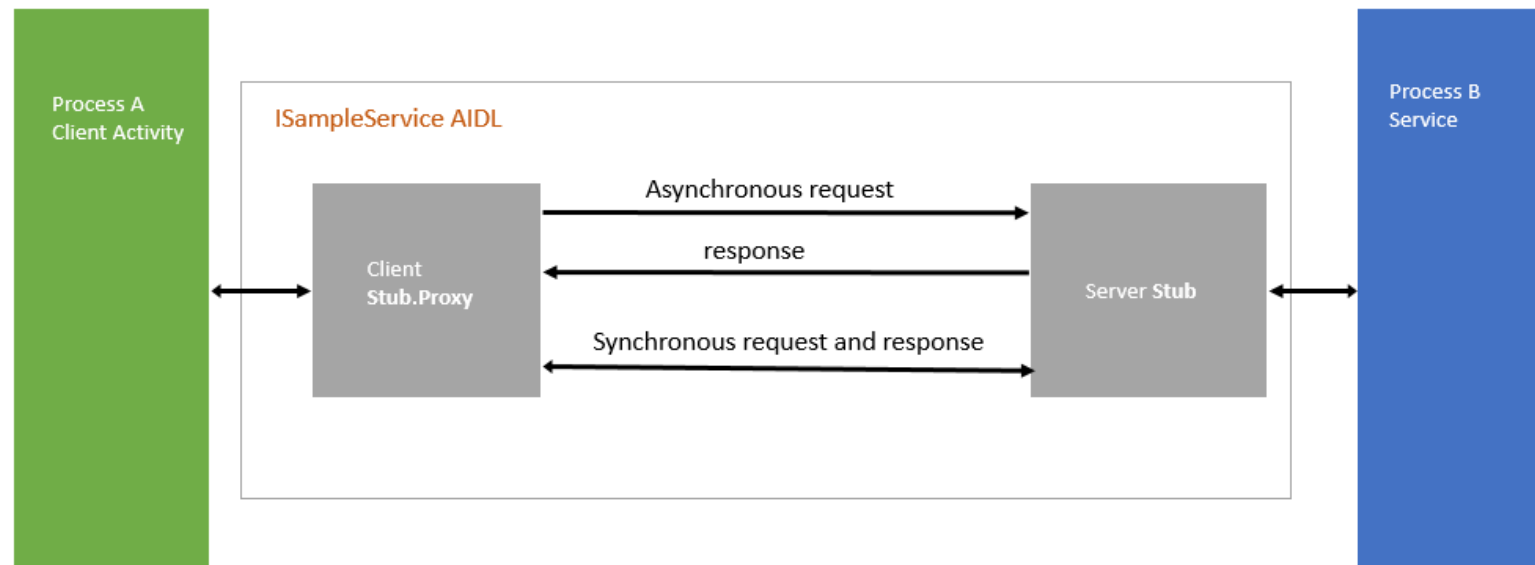
- **Expose the interface to clients**

Implement a **Service** and override **onBind()** to return your implementation of the **Stub** class.

2. IPC - AIDL (Android Interface Definition Language)

2.5. AIDL (Android Interface Definition Language)

- The Android Interface Definition Language (AIDL) is similar to other [IDLs](#): it lets you define the programming interface that both the client and service agree upon in order to communicate with each other using interprocess communication (IPC).
- On Android, one process can't normally access the memory of another process. To talk, they need to decompose their objects into primitives that the operating system can understand and marshall the objects across that boundary for you. The code to do that marshalling is tedious to write, so Android handles it for you with AIDL.



2. IPC - Messenger

- ❖ The Messenger is another IPC method in Android that relies on using Handler to send and receive messages between different processes.
- ❖ It's mainly used for sending simple messages between components of an application or between different applications.

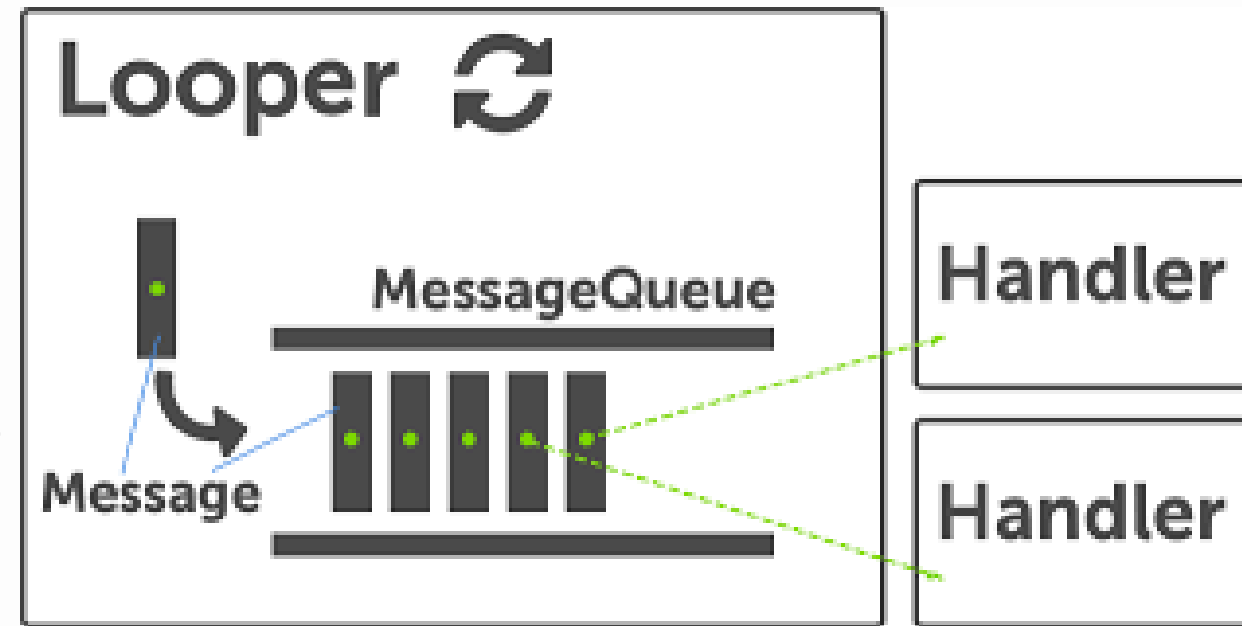
The above model is implemented in the Android via Looper, Handler, and HandlerThread. The System can be visualized to be a vehicle as in the article's cover.

MessageQueue is a queue that has tasks called messages which should be processed.

Handler enqueues task in the MessageQueue using Looper and also executes them when the task comes out of the MessageQueue.

Looper is a worker that keeps a thread alive, loops through **MessageQueue** and sends messages to the corresponding handler to process.

Finally Thread gets terminated by calling Looper's quit() method.



Demo





Thank You !

OPEN MIND - OPEN DOORS